

낮선 코드를 읽고, 증상을 재현하고, 근거로 고칩니다.

운영 중인 모바일 서비스를 개발하며 기존 코드를 분석하고 문제를 해결해 왔습니다. 지금은 C++ IOCP 서버를 직접 구현하며 네트워크와 동시성 문제를 파고들고 있습니다.

GitHub

github.com/revol7365

Portfolio repo

[game-programmer-portfolio](#)

2026

넥토리얼 게임 프로그래머

문제가 생기면, 이렇게 범위를 좁힙니다.



현상 재현

언제, 어떤 조건에서 다시 발생하는지 먼저 고정합니다.



가설 분리

초기화, 수명, 순서처럼 가능한 원인을 서로 나눕니다.



하나씩 확인

코드와 로그를 기준으로 가설을 한 번에 하나씩 검증합니다.



결과 기록

수정 전후를 같은 조건에서 비교하고 남은 한계도 함께 적습니다.

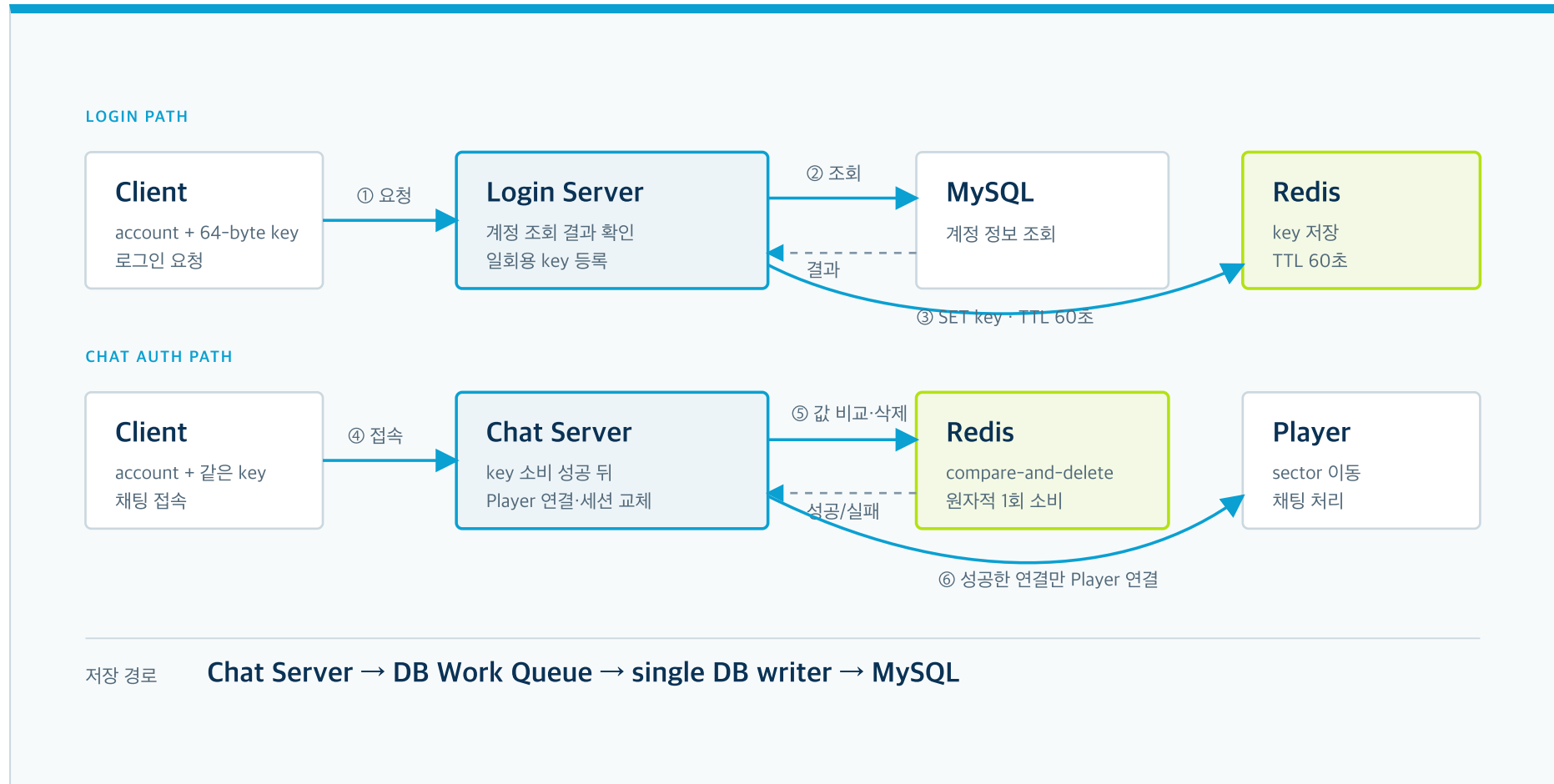
다섯 프로젝트에서 마주친 기술 문제를 하나의 흐름으로 묶었습니다.

대표 시스템에서 시작해 메모리, 동시성, 작업
분배와 소유권 문제로 범위를 넓혀 간 과정을
정리했습니다.

01	Login-Chat System C++20 · IOCP · MySQL · Redis	실패한 인증 요청이 정상 세션에 영향을 주지 않게 하려면?	대표 프로젝트
02	Memory Pool C++20 · TLS · SLIST	할당·반환 주체가 바뀌면 어떤 구조를 선택해야 하는가?	projects/memory-pool
03	Lock-Free Queue C++20 · CAS · Hazard Pointer	동시 접근 중 제거한 노드를 언제 안전하게 회수할 것인가?	projects/lock-free-queue
04	멀티스레드 채팅 서버 C++20 · IOCP · SRWLOCK	브로드캐스트에서 Lock은 어디까지만 잡아야 하는가?	projects/multithread-chat-server
05	Group Echo Server C++20 · IOCP · MPSC Queue	네트워크 작업을 그룹 단위 순서로 어떻게 처리할 것인가?	projects/group-echo-server

로그인, 인증, 채팅, 저장을 서로 다른 책임으로 나눴습니다.

현재 GitHub 코드의 세 실행 파일과 MySQL·Redis 연결을 기준으로 로그인 시스템의 전체 구조를 다시 그렸습니다.



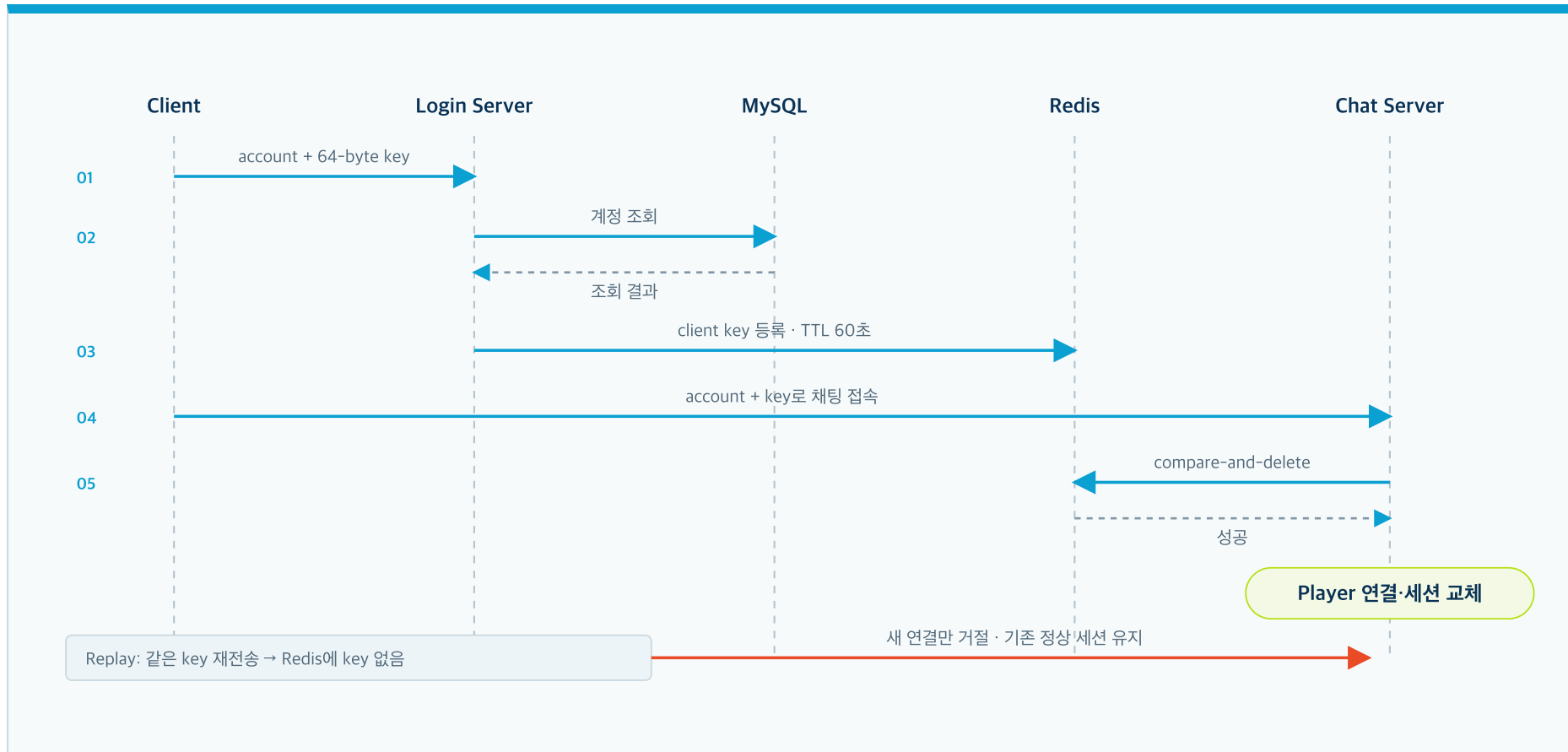
인증 경계

계정 정보는 MySQL에서 조회하고, 채팅 접속용 key는 Redis에서 60초 동안 보관한 뒤 한 번만 소비합니다.

저장 경계

게임 저장 요청은 DB Work Queue로 넘겨 single writer가 MySQL에 반영합니다. [코드 보기](#)

채팅 접속용 key는 Redis에서 한 번만 소비됩니다.



이 구조에서 고민한 것

MySQL에 인증 상태까지 계속 물으면 접속 경로가 무거워지고, Redis key를 읽기만 하면 재사용 공격을 막지 못합니다. 그래서 짧은 TTL과 원자적 1회 소비를 묶었습니다.

현재 검증

정상 key, 잘못된 key, 이미 소비한 key를 각각 보냈습니다. 실패 연결은 거절되고 기존 정상 세션의 sector 응답은 계속 도착하는지 확인했습니다.

실패한 인증 요청이, 왜 정상 사용자를 끊었을까?

재사용 key 회귀 테스트에서 이상을 발견한 뒤,
문제, 현상, 가설, 해결 순서로 원인을 좁혔습니다.

어떻게 발견했나

정상 key로 먼저 접속한 뒤 같은 계정에 이미 소비한 key를 다시 보내는 회귀 테스트를 추가했습니다. 새 연결만 거절될 것으로 예상했지만 기존 정상 연결의 다음 sector 이동 응답도 오지 않아, 인증 실패가 다른 세션에 번진다는 사실을 찾았습니다.

문제 인증 실패의 영향 범위 잘못된 연결은 그 연결만 거절해야 합니다. 인증을 통과한 기존 사용자의 세션 상태를 바꾸면 안 됩니다.	현상 정상 세션까지 종료 이미 소비했거나 잘못된 key로 같은 계정에 접속하자 새 연결은 실패했지만, 먼저 접속한 정상 세션도 끊겼습니다.	가설 검증보다 교체가 먼저 실행 기존 세션 종료가 Redis key 검증보다 앞에 있어, 인증되지 않은 연결도 중복 로그인 교체 경로에 들어간다고 봤습니다.	해결 성공 뒤에만 세션 교체 Redis compare-and-delete가 성공한 뒤 기존 세션을 찾고 교체했습니다. 실패한 요청은 새 연결만 거절했습니다.
--	--	--	--

재현 순서

정상 key로 로그인한 뒤 잘못된 key와 이미 소비한 key를 같은 계정으로 다시 보냈습니다.

통과 기준

두 실패 연결은 거절되고, 기존 정상 연결에서는 다음 sector 이동 응답이 계속 도착해야 합니다.

Memory Pool을 A, B, C 세 구조로 비교했습니다.

TLS의 부족한 batch를 어디서 가져올지 바꾸며 CAS 비용, 메모리 비용, 경합 위치를 비교했습니다.

왜 세 구조를 만들었나

빠른 TLS 경로는 같아도 refill과 drain이 한 전역 풀에 모이면 부하가 커질 수 있습니다. 전역 풀을 유지하되 CAS 횟수를 줄일지, 전역 풀 자체를 없애고 스레드끼리 균형을 맞출지 확인했습니다.

A

GlobalNode 전역 풀

TLS에서 넘긴 batch를 전역 전용 노드에 보관합니다. 전역 stack과 GlobalNode free-list를 따로 관리합니다.

특징

Node 크기는 늘리지 않지만 batch 이동마다 CAS가 두 번 필요합니다.

B

Node 직접 연결 전역 풀

일반 Node에 전역 연결 포인터를 추가해 batch 자체를 전역 stack에 연결합니다.

특징

CAS는 한 번으로 줄지만 모든 Node가 포인터 하나를 더 가집니다.

C

스레드 간 batch 이동

전역 풀을 거치지 않고 batch가 가장 많이 남은 스레드에서 가져옵니다.

특징

중앙 경합과 TLS 편중을 줄이지만 thread index와 다른 TLS 잔량을 조회합니다.

실험 환경

Xeon E5-2608 v4, 14코어 28스레드, RAM 16GB, 데이터 1KB, 스레드당 10,000회 할당·해제를 100회 반복했습니다. high_resolution_clock으로 전 평균 경과 시간은 ms 단위입니다.

연구 질문

단일 스레드, 같은 스레드 반환, 다른 스레드 반환으로 조건을 바꾸면 가장 빠른 구조도 바뀌는지 확인했습니다.

과거 고부하에서는 C가 빨랐고, 현재는 객체 수명을 우선했습니다.

1KB 객체 할당·반환 평균 경과 시간 단위 ms · 100회 평균 · 낮을수록 빠름

실행 조건	작업 분담	new/delete	가장 낮은 Pool
단일 스레드	1개가 할당·반환	0.729	B LF 0.077
112스레드 같은 스레드 반환	각자 할당·반환	8.717	C CS 3.671
112스레드 교차 반환	56개 할당 56개 반환	492.838	C LF 307.577

아래 표는 1KB 객체를 반복 할당·반환했을 때 한 회차를 끝내는 평균 경과 시간(ms)입니다. 숫자가 낮을수록 빠릅니다.

저경합

단일 스레드에서는 CAS를 한 번으로 줄인 B Lock-Free가 0.077로 가장 낮았습니다.

같은 스레드 반환

112스레드에서는 C CS 3.671과 C Lock-Free 3.681이 거의 같았습니다.

교차 반환

할당과 반환 스레드를 나눈 조건에서는 C Lock-Free가 307.577로 가장 낮았습니다.

내 결론

A와 B의 전역 풀보다 C의 스레드 간 batch 이동이 고부하에 맞았습니다. 여유 batch 편중은 줄지만 잔여 노드가 완전히 없어지는 구조는 아닙니다.

과거 성능 선택

구조 C는 전역 병목을 없애고, 부족한 스레드가 가장 많은 batch를 가진 스레드에서 가져오도록 했습니다.

현재 공개본의 선택

종료 시 회수되지 않던 3,200개 객체를 확인한 뒤 성능보다 수명을 우선했습니다. Windows가 제공하는 원자적 단일 연결 리스트(SLIST)를 전역 free-list로 사용했고, 16 threads × 100,000회 뒤 live object 0을 확인했습니다. **테스트 기록**

같은 Memory Pool 구조에서도 반환 방식에 따라 결과가 달랐습니다.

A/B/C 구조를 유지하고 공유 지점의 동기화만 바꿔, 같은 스레드 반환과 교차 반환에서 결과가 어떻게 달라지는지 비교했습니다.

무엇을 고정했나

A/B/C 각각의 Pool 구조, 1KB 객체, 스레드당 10,000회 할당·반환, 100회 반복은 같게 두었습니다.

무엇만 바꿨나

각 구조에서 전역 공유 지점을 CRITICAL_SECTION으로 보호한 버전과 CAS 기반 Lock-Free 버전을 교체했습니다.

어떤 부하를 줬나

112개가 각자 반환하는 조건과 56개 할당·56개 반환 조건을 비교했습니다. 교차 조건에는 공통 전달 Queue 비용도 포함됩니다.

왜 CS와 Lock-Free를 비교했나

Lock-Free는 대기과 context switch를 줄일 수 있지만 CAS 재시도와 메모리 접근 비용이 생깁니다.

그래서 구현 이름으로 판단하지 않고 같은 Pool 구조에서 동기화 방식만 바꿨습니다.

같은 스레드 반환과 교차 반환을 나눠, 경쟁 위치가 바뀌면 결과도 달라지는지 확인했습니다.

구조 / 평균(ms)	112T 같은 반환 LF	112T 같은 반환 CS	112T 교차 반환 LF	112T 교차 반환 CS
A	8.948	9.163	337.254	375.178
B	8.890	9.519	324.279	350.639
C	3.681	3.671	307.577	337.300

같은 스레드 반환

C의 CS 3.671ms와 Lock-Free 3.681ms는 0.010ms, 약 0.3% 차이였습니다. 분산값이 없어 승패로 해석하지 않고 사실상 동률로 보았습니다.

교차 반환

A/B/C 모두 Lock-Free 평균 경과 시간이 낮았습니다. 이 결과는 Memory Pool 공유 지점에 한정하며 Queue 성능의 근거로 사용하지 않았습니다. 근거: 직접 작성한 「MemoryPool 테스트.xlsx」

Queue에서는 처리 시간보다 누락, 중복, 손상을 먼저 확인했습니다.

다중 생산자와 소비자가 같은 노드를 바꾸는
동안 연결 불변식과 노드 수명이 함께
유지되는지 검증했습니다.

8 + 8 Producer 8개와 Consumer 8개를 동시에 실행	1,600,000 한 회차에서 생산하고 소비한 item 수	10회 Debug와 Release에서 동일 조건 반복	0 / 0 / 0 missing, duplicate, corrupt
--	--	---	---

왜 MPMC 경계 검증 8개 Producer와 8개 Consumer가 head, tail, next를 동시에 바꿀 때도 FIFO와 정확한 1회 소비가 필요했습니다.	문제 컴파일 뒤에도 데이터 손실 CAS 캐스팅을 고친 뒤에도 누락과 size 불일치가 남아 연결과 회수 문제를 따로 확인했습니다.	해결 Hazard Pointer 지연 회수 head와 next를 보호하고 제거 노드는 retired list에 둔 뒤 활성 hazard에서 사라질 때 회수했습니다.	결론 조건부로 사용 누락·중복·손상 0을 확인한 MPMC 경계에만 사용합니다. 단일 consumer Queue에는 더 단순한 Lock을 유지합니다.
---	---	---	--

Content Thread 1개를 4개로 나누고 세션별 작업 순서를 유지했습니다.

동일 장비와 동일한 더미 클라이언트 설정에서 Content Thread 수만 1개에서 4개로 바꿔 비교했습니다. 표는 5,000·10,000·15,000명 테스트 중 15,000명 구간입니다.

초기 구조 · Content Thread 1개

Join·Leave·Recv가 하나의 Content Queue로 들어가고, 한 스레드가 Player 상태 변경을 순서대로 처리했습니다.

변경 구조 · Content Thread 4개

CONTENT_THREAD_CNT = 4 · SessionID % 4로 담당 스레드를 고정하고, Join·Leave·Recv Job을 각 IOCP Queue에 PostQueuedCompletionStatus(PQCS)로 전달했습니다.

15,000명 구간 발체	Accept TPS	Recv TPS	Send TPS	더미 1/2/3 평균 응답(ms)
싱글	2,282	5,239	24,267	285 / 1,193 / 1,188
멀티	4,451	122,064	986,477	10 / 7 / 3

문제를 찾은 단서

Error Tracking 기록에는 로그를 줄인 뒤에도 장시간 지연이 남았습니다. 동일 장비와 부하 설정에서 Content Thread 수만 독립 변수로 두고 TPS와 응답 시간을 비교했습니다.

가설과 변경

한 consumer에 모인 Player 상태 변경을 병목 후보로 두고, 담당 Thread를 세션별로 고정해 네 Queue로 분산했습니다.

결과와 해석

동일 조건의 멀티 구성에서 TPS가 높아지고 세 더미의 평균 응답 지연이 낮아졌습니다. 단일 Content Queue의 직렬 처리가 병목에 영향을 주고 있음을 확인했고, Thread별 부하 편중은 새 비용으로 남겼습니다.

구조 근거

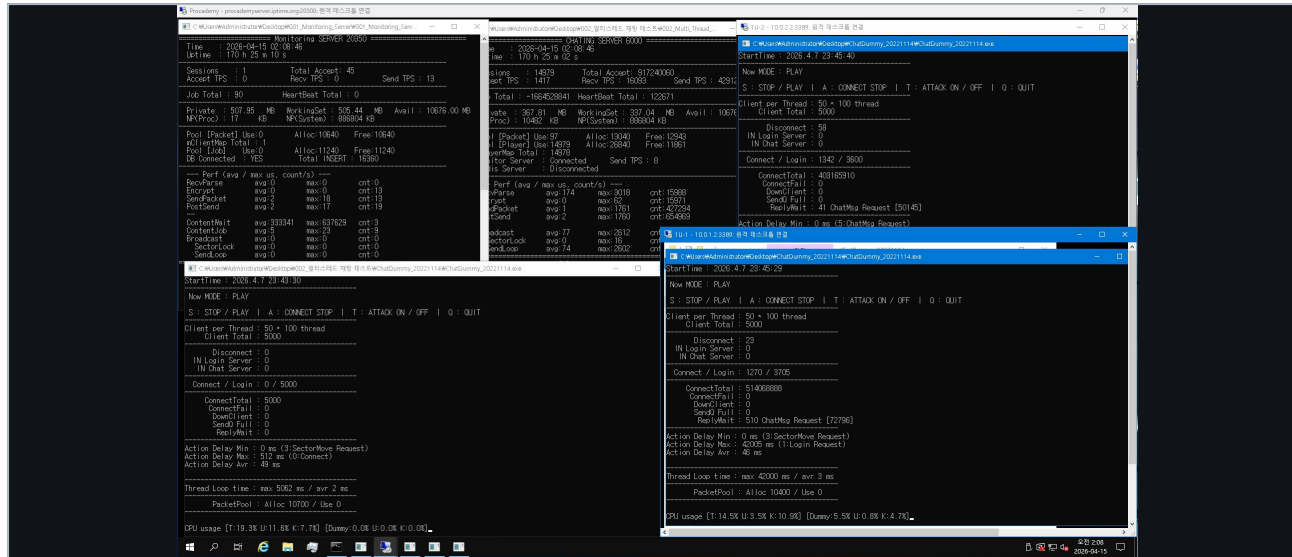
초기 설계 노트와 single 코드는 Queue·Content Thread 1개를 보여 줍니다. muti-main에는 상수 4, SessionID % 4, PQCS 전달이 구현돼 있습니다.

측정 근거와 한계

수치는 직접 작성한 「채팅 서버 멀티 스레드 테스트.xlsx」의 원본 값입니다. 동일 장비에서 비교했지만 세부 사양·측정 시간·반복 횟수는 기록이 없어 현재 환경의 성능 보증값으로 사용하지 않습니다. 전체 기록

약 7일 가동 시점에, 14,979개 세션을 확인했습니다.

날짜, 서버 Uptime, Sessions가 함께 보이는
멀티스레드 채팅 서버 화면을 장시간 관측
근거로 사용했습니다.



2026-04-15 02:08:46 · Uptime 170시간 25분 02초 · Sessions 14,979.

가동 시점

170시간 25분은 약 7일 2시간입니다. 화면 날짜와 함께 확인됩니다.

접속 상태

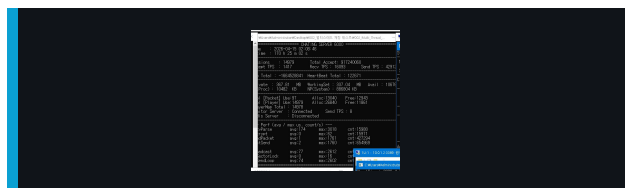
Sessions와 Player Use가 각각 14,979로 표시됩니다.

처리량-메모리

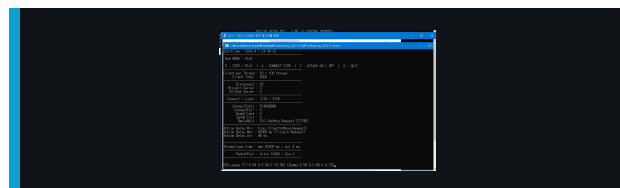
Accept 1,417, Recv 16,093, Send 42,912 TPS와 WorkingSet
337.04MB가 보입니다.

사용자 관점

함께 실행한 더미 화면 한 대에서 Client 5,000명과 Action Delay 평균
46ms를 확인했습니다.



WorkingSet 337.04MB · Recv 16,093 · Send 42,912 TPS



Dummy 5,000명 화면 · Action Delay 평균 46ms

이 화면은 약 7일 가동 시점의 관측입니다. 14,979개 세션을 7일 내내 유지했다는 뜻으로 확대하지 않았습니다. 화면의 음수 Job Total은 32비트 누적 카운터가 범위를 넘어 표시된 값이며, 서버 기능 실패와 구분해 성능 지표에서 제외했습니다.

현재 공개본에서는 브로드캐스트의 Lock 안에서 대상만 수집합니다.

과거 부하 실험과 구분해, 현재 GitHub 코드에서 확인되는 Lock 경계와 Player 수명 처리 방식을 설명합니다.

IOCP Worker → OnRecv

packet framing, checksum, 암호호화를 거친 뒤 type과 payload 길이를 확인합니다.

Player map은 Shared Lock으로 조회하고, 잘못된 account·sector·message length는 거절합니다.



Sector 조회

현재 섹터와 주변 최대 9개 섹터를 미리 계산
각 Sector Shared Lock 안에서는 SessionID만 수집

Broadcast

Sector Lock을 모두 놓은 뒤
SendPacket 반복
Sector Lock과 송신 경로의 Lock 중첩을 피함

Leave

Player map에서 먼저 제거하고
WillDelete 표시
Sector 정리와 pool 반환은 map Lock 밖에서 수행

고민 1 · Lock 범위

9개 섹터를 잠금 상태로 모든 세션에 송신하면 fan-out만큼 Lock 보유 시간이 늘어납니다.

고민 2 · 해제 순서

떠나는 Player를 먼저 검색 대상에서 제외한 뒤 Sector와 Pool을 정리해 새 broadcast가 잡지 않게 했습니다.

검증

16 login, 16 sector move, 256 chat response에서 type, account, body, checksum을 대조했습니다.

코드 근거: ChatServer.cpp의 broadcast는 Sector Lock 안에서 대상 ID만 모으고, Lock 밖에서 SendPacket을 호출합니다. [코드 보기](#)

일반 채팅 서버를 그룹 처리 구조로 바꾼 이유가 있습니다.

한 Content Queue에 모든 작업을 모으면
느린 작업 하나가 무관한 사용자까지
기다리게 합니다. 반대로 모두 병렬화하면
같은 상태의 순서를 설명하기 어렵습니다.

하나의 전역 Queue

모든 콘텐츠를 한 스레드가 순서대로 처리합니다.

장점 순서가 단순합니다.

문제 느린 그룹 하나가 전체를 막습니다.

상태마다 Lock

여러 스레드가 Player와 그룹 상태를 동시에 수정합니다.

장점 세밀한 병렬화가 가능합니다.

문제 Lock 순서와 입장·퇴장 순서가 복잡해집니다.

그룹별 Queue

같은 그룹은 단일 consumer가 직렬 처리하고, 다른 그룹은
독립적으로 실행합니다.

선택 이유 그룹 내부 순서와 그룹 간 병렬성을 서로 다른 경계로
나눌 수 있습니다.

Network Worker
패킷 검증·Job 생성



Login Group
인증·Player 생성



Echo Groups
그룹별 단일 consumer

이 프로젝트는 그룹 채팅 구조의 핵심인 라우팅, 그룹 내부 직렬 처리, 그룹 이동 순서를 Echo 요청으로 검증한 프로토타입입니다. [코드 보기](#)

그룹 처리의 핵심은 Queue보다 Player 소유권 순서였습니다.

그룹별 단일 consumer만으로는 이동 중 안전이 보장되지 않습니다. GROUP_ENTER, 소유 그룹 포인터, 늦은 LEAVE의 순서를 함께 정했습니다.

문제가 되는 순서

Player의 소유 그룹을 먼저 바꾸면, 이전 Login Group에 이미 들어온 LEAVE가 Player를 해제할 수 있습니다. 새 Echo Group의 GROUP_ENTER는 해제된 포인터를 받게 됩니다.



선택한 순서

대상 Echo Group mailbox에 GROUP_ENTER를 먼저 넣고, 그다음 mGroup을 release-store로 교체했습니다. 이후 늦은 작업은 새 그룹으로 전달됩니다.



정리 책임

이전 그룹의 onLeave는 현재 mGroup이 자신이 아니면 LEAVE를 새 그룹에 다시 넣습니다. Player 반환은 현재 소유 그룹만 수행합니다.

특징 1 · 그룹 내부 직렬화

각 Group의 mPlayers는 Content Thread 하나만 수정해 같은 그룹의 상태 변경 순서를 한곳에서 보장합니다.

특징 2 · 그룹 간 병렬성

Login Group과 여러 Echo Group은 서로 다른 mailbox와 consumer를 가져 독립적으로 실행합니다.

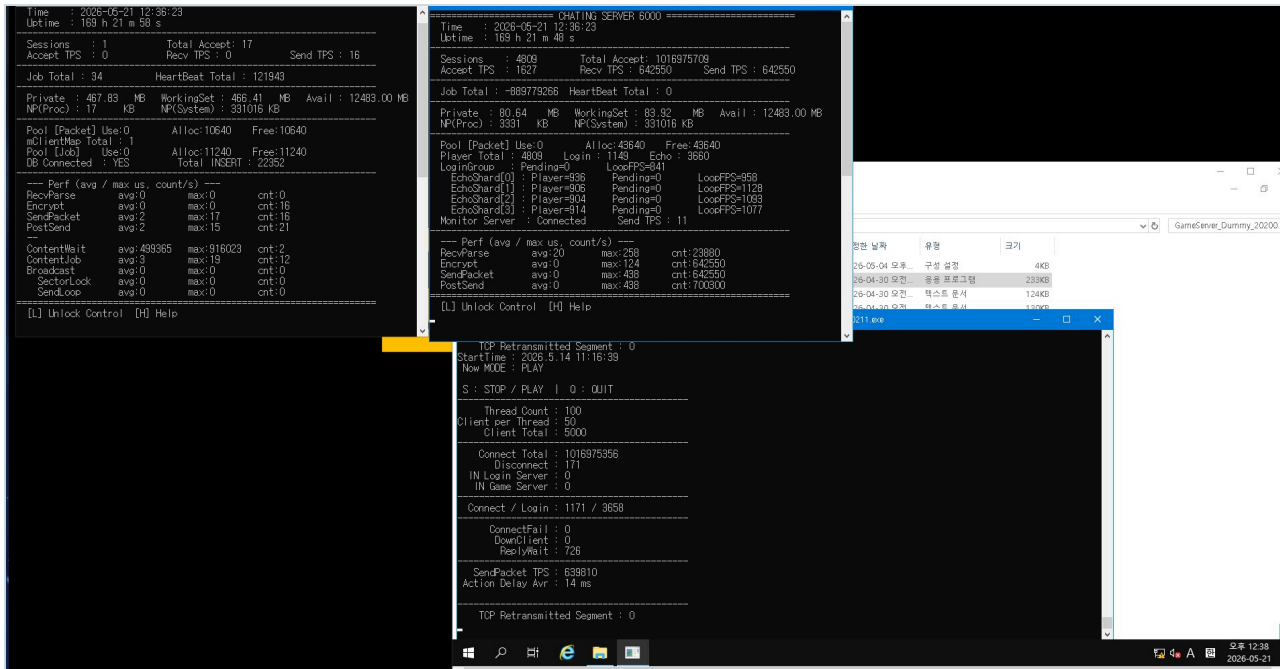
특징 3 · 단순한 mailbox

이동 검증과 ABA·spin 위험을 섞지 않기 위해 MPSC mailbox는 CRITICAL_SECTION과 std::queue를 사용했습니다.

해결 순서: 대상 mailbox에 GROUP_ENTER를 먼저 넣고 mGroup을 교체했습니다. 이전 그룹에 늦게 도착한 LEAVE는 현재 소유 그룹으로 다시 전달했습니다. [코드 보기](#)

기능 테스트와 장시간 화면으로 그룹 분산과 Queue 상태를 확인했습니다.

현재 공개본의 반복 기능 검증과 과거 장시간
실행 화면을 구분해, 정확성과 실행 상태를
각각 확인했습니다.



2026-05-21 12:36:23 · Uptime 169시간 21분 48초 · Sessions 4,809.

16 clients × 10 echo

로그인 16건과 account·tick을 대조한 응답 160건이 모두 통과했습니다.

Echo Player 3,660

과거 화면에서 Shard별 936 / 906 / 904 / 914로 분산돼 있습니다.

Pending 0

Login Group과 네 EchoShard의 화면상 대기 Job이 모두 0입니다.

Action Delay 14ms

함께 실행한 Dummy 화면의 평균 관측값입니다.

내 결론: 같은 그룹의 순서는 단일 consumer로 단순하게 만들고, 서로 무관한 그룹만 병렬로 실행하는 구조가 적합했습니다. 기능 테스트는 현재 공개본 결과이며, 169시간 화면은 특정 시점 관측입니다. 화면의 음수 Job Total은 32비트 누적 카운터의 표시 한계로 성능 지표에서 제외했습니다.